

# نظریه زبان ها و ماشین ها

دانشگاه فردوسی مشهد

گروه مهندسی کامپیوتر

دکتر سوادى

نیمسال دوم ۱۴۰۵-۱۴۰۴

پروژه شماره ۲ - Checker Model

مهلت تحویل: ۱۴۰۵/۴/۲۶ ۲۳:۵۹

مسئولین پروژه:

|              |                  |                      |
|--------------|------------------|----------------------|
| مسئول پروژه: | @Amir_hosseini83 | (امیرحسین ابوالفضلی) |
| مسئول پروژه: | @arman_bjr       | (آرمان بیجاری)       |

## راهنمای تحویل و نکات مهم پروژه

### نکات تحویل

- فاز اول: سورس کد ترنسلیتور و فایل pml. خروجی در پوشه Phase1-Promela.
- فاز دوم: فایل ltl. حاوی فرمول‌های سه property و خروجی کامل SPIN در پوشه Phase2-Verification.
- فاز سوم: فایل اثبات با پسوند lean. یا v. بسته به ابزار انتخابی در پوشه Phase3-Proof.
- گزارش (PDF): توضیح تصمیم‌های طراحی، نگاشت‌ها، فرمول‌های LTL، خروجی ترمینال و تفسیر منطق اثبات.

### ساختار پوشه‌ها:

StudentID-FullName-Project2-ModelChecker.zip

Format

StudentID-FullName-Project2.zip

```
├── Report/
│   └── Report.pdf
├── Phase1-Promela/
│   ├── translator_src/
│   └── output.pml
├── Phase2-Verification/
│   ├── properties.ltl
│   └── spin_results/
├── Phase3-Proof/
│   └── proof.lean OR proof.v
```

### نمره‌دهی و تسلط

پروژه در قالب تیم‌های دونفره قابل انجام است. هرگونه کپی‌برداری از کد، مدل، یا اثبات دیگران مصداق تقلب است و مطابق قوانین درس با آن برخورد خواهد شد. استفاده از ابزارهای هوش مصنوعی صرفاً در حد کمک آموزشی مجاز است و طراحی نهایی مدل، property ها و اثبات‌ها باید حاصل کار و درک شخصی خود دانشجو باشد. تقسیم‌بندی نمره به صورت زیر است:

• فاز اول – ترجمه به Promela: 30%

• فاز دوم – Model Checking: 40%

• فاز سوم – اثبات رسمی: 30%

## فهرست مطالب

|    |       |                                  |
|----|-------|----------------------------------|
| ۳  | ۱     | مقدمه                            |
| ۳  | ۲     | منابع مطالعاتی                   |
| ۴  | ۳     | فاز اول – ترجمه به Promela       |
| ۴  | ۱.۳   | معرفی SPIN و Promela             |
| ۴  | ۲.۳   | زیرمجموعه Hash مورد نظر          |
| ۴  | ۳.۳   | قوانین ترجمه                     |
| ۵  | ۴.۳   | نمونه                            |
| ۵  | ۴     | فاز دوم – Model Checking با SPIN |
| ۵  | ۱.۴   | معرفی LTL و ارتباط با اتوماتا    |
| ۶  | ۲.۴   | خصوصیات تعریف شده                |
| ۶  | ۱.۲.۴ | Safety                           |
| ۶  | ۲.۲.۴ | Liveness                         |
| ۶  | ۳.۲.۴ | Deadlock Freedom                 |
| ۷  | ۵     | فاز سوم – اثبات رسمی با Coq/Lean |
| ۷  | ۱.۵   | معرفی Operational Semantics      |
| ۷  | ۲.۵   | معناشناسی عملیاتی زیرمجموعه Hash |
| ۸  | ۳.۵   | مثال راهنما – اثبات Determinism  |
| ۱۰ | ۴.۵   | وظیفه دانشجوی                    |

# توضیحات و بخش‌های پروژه

## ۱. مقدمه

توی پروژه اول، زبان Hash رو از صفر طراحی کردید - گرامر نوشتید، token ها رو تعریف کردید، و parse tree ساختید. حالا می‌خوایم به قدم عمیق‌تر بریم و از این زبان سوال بپرسیم. سوال‌هایی که خیلی مهم‌تر از «آیا این کد syntax درستی داره؟» هستن.

مثلاً: آیا ممکنه برنامه‌ای که به زبان Hash نوشته شده هیچ‌وقت به یه حالت خطرناک برسه؟ آیا یه حلقه حتماً یه روزی تموم می‌شه؟ آیا می‌شه ثابت کرد که یه برنامه درست کار می‌کنه - نه که فقط تستش کرد؟ اینا قلب Formal Verification هستن، و جواب دادن بهشون دقیقاً هدف این پروژه‌ست.

### ارتباط با نظریه اتوماتا

شاید بپرسید این بحث چه ربطی به درس «نظریه زبان‌ها و ماشین‌ها» داره. ربطش مستقیم و جالبه. یه برنامه در حال اجرا رو می‌شه به عنوان یه state machine در نظر گرفت - هر وضعیت حافظه یه state ه، و هر دستور یه transition. Model Checking دقیقاً همین state machine رو می‌گیره و روش خصوصیات رو چک می‌کنه که با LTL (Linear Temporal Logic) بیان شدن. جالب اینجاست که LTL خودش مستقیماً به Buchi automaton ترجمه می‌شه - که یه مفهوم کلاسیک توی نظریه اتوماتاست.

کار این پروژه توی سه فاز اصلی انجام می‌شه. توی فاز اول، یه زیرمجموعه از Hash رو به Promela ترجمه می‌کنید - زبان ورودی ابزار SPIN - و می‌تونید از همون Visitor/Listener هایی که توی پروژه اول نوشتید کمک بگیرید. توی فاز دوم، روی مدل ترجمه‌شده سه property مشخص رو با استفاده از SPIN بررسی می‌کنید. توی فاز سوم، یه قدم عمیق‌تر می‌رید: operational semantics زبان رو تعریف می‌کنید و یه property رو به صورت کاملاً رسمی توی Lean یا Coq اثبات می‌کنید.

## ۲. منابع مطالعاتی

قبل از اینکه وارد فازها بشید، لازمه با چند تا مفهوم و ابزار جدید آشنا بشید. این بخش یه راهنمای مطالعاتیه.

- **LTL و CTL** - اگه می‌خواید بفهمید model checking زیر هودش چطور کار می‌کنه، باید با temporal logic آشنا بشید. بهترین منبع برای شروع فصل سوم کتاب Principles of Model Checking نوشته Baier & Katoen (MIT Press, 2008) ه. مفهوم Buchi automaton هم توی همین فصل توضیح داده می‌شه.
- **SPIN و Promela** - ابزار اصلی فاز دومه. برای شروع، سایت رسمی spinroot.com بهترین جاست - یه tutorial ساده و خوب داره که در عرض چند ساعت می‌تونید باهاش Promela رو یاد بگیرید.
- **Operational Semantics** - برای فاز سوم باید بدونید operational semantics چیه. فصل دوم کتاب The Formal Semantics of Programming Languages نوشته Glynn Winskel (MIT Press, 1993) (نشر MIT Press) نقطه شروع خوبیه.
- **Lean** - اگه proof assistant انتخابیتون Lean 4 ه، کتاب Mathematics in Lean بهترین جا برای شروعه. به صورت رایگان روی [leanprover-community.github.io](https://leanprover-community.github.io) در دسترسه.
- **Coq** - اگه Coq رو ترجیح می‌دید، (Software Foundations Vol. 1 (Logical Foundations) نوشته Pierce et al. استانداردترین منبع آموزشی Coq ه. به صورت رایگان روی [softwarefoundations.cis.upenn.edu](https://softwarefoundations.cis.upenn.edu) هست.

### یه پیشنهاد

لازم نیست همه این منابع رو از اول تا آخر بخونید. اصلاً شاید حتی نیاز نباشه برید بخونید و با یه سرچ ساده متوجه مطلب بشید. علاوه بر این هر فاز توی این سند با معرفی کافی شروع می‌شه، ولی اگه می‌خواید یه درک عمیق‌تر داشته باشید حتماً نگاه کنید. جالبه که بدونید خیلی از شرکت‌های دنیا مثل Amazon الان در حال سرمایه گذاری میلیون دلاری روی اثبات درست بودن نرم‌افزارهاشون با استفاده از ابزارهای مورد استفاده توی این پروژه هستن.

## ۳. فاز اول – ترجمه به Promela

### هدف این فاز

در این فاز باید یه ترنسلیتور بنویسید که برنامه‌های Hash رو به Promela تبدیل کنه. تنها چیزی که تحویل می‌دید سورس کد ترنسلیتور و فایل pml. خروجیه.

### ۱.۳ معرفی SPIN و Promela

SPIN یه ابزار model checking ه که توسط Gerard Holzmann در آزمایشگاه‌های Bell Labs توسعه پیدا کرده. ورودی SPIN به زبانی به اسم Promela (Process Meta Language) نوشته می‌شه.

### ۲.۳ زیرمجموعه Hash مورد نظر

برای این پروژه لازم نیست کل زبان Hash رو ترجمه کنید. زیرمجموعه‌ای که باید پشتیبانی کنید شامل موارد زیره:

- متغیرهای از نوع adad و boole
- دستور شرطی age/bood/vagarna
- حلقه‌های ta و baraye به همراه shekan و edame
- عملگرهای محاسباتی و مقایسه‌ای تعریف شده در پروژه اول

### محدودیت‌های زیرمجموعه

چند نکته درباره دقیق محدوده پشتیبانی: shekan فقط از حلقه‌ای که مستقیماً داخلش خارج می‌شه، نه از حلقه‌های بیرونی‌تر. edame اجرای بدنه فعلی رو متوقف می‌کنه و به شرط حلقه برمی‌گرده. کلاس‌ها و method ها هم جزو این زیرمجموعه نیستن.

### ۳.۳ قوانین ترجمه

جدول زیر نگاشت اصلی از Hash به Promela رو نشون می‌ده.

| Promela                                               | Hash                                    |
|-------------------------------------------------------|-----------------------------------------|
| int x = 5                                             | adad x = 5                              |
| bool b = true                                         | boole b = dorost                        |
| bool b = false                                        | boole b = ghalat                        |
| if :: (cond) -> ... :: else -> ... fi                 | age (cond) bood { ... } vagarna { ... } |
| do :: (cond) -> ... :: else -> break od               | ta (cond) { ... }                       |
| init; do :: (cond) -> ...; update :: else -> break od | baraye (init; cond; update) { ... }     |
| break                                                 | shekan                                  |
| do/od label به ابتدای                                 | edame                                   |

چند نکته مهم درباره این نگاشت: برای edame، باید به label به اسم loop\_start قبل از بدنه حلقه تعریف کنید و با goto loop\_start بهش برگردید. در مورد baraye، دستور init قبل از do/od میاد و دستور update آخر بدنه داخل حلقه قرار می‌گیره.

### ۴.۳. نمونه

برنامه زیر به زبان Hash به حلقه ساده داره که مجموع اعداد 1 تا 5 رو حساب می‌کنه:

Hash

```
1      adad i = 1;
2      adad sum = 0;
3
4      ta (i <= 5) {
5          sum = sum + i;
6          i = i + 1;
7      }
```

معادل این برنامه در Promela به صورت زیره:

Promela

```
1      int i = 1;
2      int sum = 0;
3
4      active proctype main() {
5          do
6              :: (i <= 5) ->
7                  sum = sum + i;
8                  i = i + 1
9              :: else -> break
10         od
11     }
```

## ۴. فاز دوم – Model Checking با SPIN

### هدف این فاز

در این فاز باید مدل Promela از فاز اول رو بگیرید و سه property مشخص رو روش با SPIN چک کنید. تنها چیزی که تحویل می‌دید فایل .ltl. و خروجی SPIN برای هر property ه.

### ۱.۴. معرفی LTL و ارتباط با اتوماتا

زبانی که باهاش خصوصیات مدل را بررسی می‌کنیم (Linear Temporal Logic) LTL ه. LTL بهتون اجازه می‌ده درباره رفتار یه سیستم در طول زمان حرف بزنید. چهار اپراتور اصلی LTL که توی این پروژه باهاشون کار می‌کنید اینان:

| معنی                                    | اپراتور           |
|-----------------------------------------|-------------------|
| در تمام لحظات اجرا، $p$ برقراره         | $\Box p$          |
| در یه لحظه‌ای از اجرا، $p$ برقرار می‌شه | $\Diamond p$      |
| $p$ برقراره تا وقتی که $q$ برقرار بشه   | $p \mathcal{U} q$ |
| در قدم بعدی اجرا، $p$ برقراره           | $\bigcirc p$      |

ارتباط LTL با اتوماتا مستقیم و جالبه. SPIN وقتی یه فرمول LTL بهش می‌دید، اون رو به یه Buchi automaton تبدیل می‌کنه که دقیقاً همون دنباله‌های اجرایی رو می‌پذیره که فرمول روشن صادقه.

### Buchi automaton چیه؟

یه Buchi automaton دقیقاً مثل یه NFA معمولیه، با یه فرق: روی کلمات بی‌نهایت کار می‌کنه. یه کلمه بی‌نهایت رو می‌پذیره اگه اجرای اتوماتا روی اون کلمه بی‌نهایت بار از یه حالت پذیرنده رد بشه. به عبارت دیگه، اگه  $F$  مجموعه حالت‌های پذیرنده باشه، یه اجرای بی‌نهایت رو می‌پذیریم اگه:

$$\inf(\rho) \cap F \neq \emptyset$$

## ۲.۴. خصوصیات تعریف‌شده

در این بخش سه property مشخص تعریف شده که باید روی مدل Promela تون بررسی کنید.

### ۱.۲.۴ Safety

$$\Box !(\text{divByZero})$$

این property می‌گه که تقسیم بر صفر هیچ‌وقت توی برنامه اتفاق نمی‌افته. یه boolean flag به اسم `divByZero` تعریف کنید و قبل از تقسیم چکش کنید.

Terminal

```
spin -search -ltl "[!divByZero]" model.pml
```

### ۲.۲.۴ Liveness

$$\Box (\text{inLoop} \rightarrow \Diamond \text{exitLoop})$$

این property می‌گه که هر بار وارد یه حلقه می‌شیم، بالاخره از اون حلقه خارج می‌شیم. دو label به اسم `inLoop` و `exitLoop` در بدنه و بعد از خروج حلقه قرار دهید.

Terminal

```
spin -search -ltl "[inLoop -> <>exitLoop]" model.pml
```

### ۳.۲.۴ Deadlock Freedom

این property نیاز به فرمول LTL نداره. SPIN به صورت توکار می‌تونه deadlock رو چک کنه.

```
spin -search -deadlock model.pml
```

### انتخاب برنامه مناسب

برای اینکه سه property بالا همه معنی‌دار باشن، برنامه Hash ای رو انتخاب کنید که: حداقل یه عملیات تقسیم داشته باشه (برای safety) و حداقل یه حلقه داشته باشه (برای liveness). برنامه‌ای که خیلی ساده‌ست ارزش بررسی نداره.

## ۵. فاز سوم – اثبات رسمی با Coq/Lean

### هدف این فاز

در این فاز باید operational semantics زیرمجموعه Hash رو پیاده کنید و قضیه مشخص شده در بخش آخر رو اثبات کنید. تنها چیزی که تحویل می‌دید فایل lean. یا v. ه که کامپایل بشه.

### ۱.۵. معرفی Operational Semantics

Operational semantics یه روش رسمی برای تعریف معنای یه زبان برنامه‌نویسیه. به جای اینکه بگید گرامر زبان چیه، می‌گید هر دستور چطور اجرا می‌شه. ما از big-step semantics استفاده می‌کنیم. نماد اصلی اینه:

$$\langle S, \sigma \rangle \Downarrow \sigma'$$

که می‌خونیمش: «دستور  $S$  در محیط  $\sigma$  اجرا می‌شه و محیط  $\sigma'$  رو تولید می‌کنه.»

### ۲.۵. معنانشناسی عملیاتی زیرمجموعه Hash

قوانین ارزیابی عبارات:

$$\begin{array}{c} \overline{\langle n, \sigma \rangle \Downarrow n} \quad (\text{Num}) \qquad \overline{\langle x, \sigma \rangle \Downarrow \sigma(x)} \quad (\text{Var}) \\[10pt] \frac{\langle e_1, \sigma \rangle \Downarrow v_1 \quad \langle e_2, \sigma \rangle \Downarrow v_2}{\langle e_1 + e_2, \sigma \rangle \Downarrow v_1 + v_2} \quad (\text{Add}) \end{array}$$



### ۳.۵. مثال راهنما – اثبات Determinism

می‌خواهیم ثابت کنیم که معنانشناسی زبان Hash قطعی.

$$\langle S, \sigma \rangle \Downarrow \sigma' \wedge \langle S, \sigma \rangle \Downarrow \sigma'' \Rightarrow \sigma' = \sigma''$$

#### تعریف انواع پایه

Lean 4

```
1      abbrev Var    := String
2      abbrev State := Var -> Int
3
4      inductive Expr
5      | num : Int    -> Expr
6      | var : Var    -> Expr
7      | add : Expr   -> Expr -> Expr
8      | lt  : Expr   -> Expr -> Expr
9
10     inductive Stmt
11     | assign : Var  -> Expr -> Stmt
12     | seq    : Stmt -> Stmt -> Stmt
13     | ifElse : Expr -> Stmt -> Stmt -> Stmt
14     | while  : Expr -> Stmt -> Stmt
```

Coq

```
1      Require Import Coq.Strings.String.
2      Require Import Coq.ZArith.ZArith.
3
4      Definition Var    := string.
5      Definition State := Var -> Z.
6
7      Inductive Expr : Type :=
8      | ENum : Z    -> Expr
9      | EVar : Var  -> Expr
10     | EAdd : Expr -> Expr -> Expr
11     | ELt  : Expr -> Expr -> Expr.
12
13     Inductive Stmt : Type :=
14     | SAssign : Var  -> Expr -> Stmt
15     | SSeq    : Stmt -> Stmt -> Stmt
16     | SIf     : Expr -> Stmt -> Stmt -> Stmt
17     | SWhile  : Expr -> Stmt -> Stmt.
```

Lean 4

```

1      def evalExpr (e : Expr) (s : State) : Int :=
2      match e with
3      | Expr.num n      => n
4      | Expr.var x      => s x
5      | Expr.add e1 e2 => evalExpr e1 s + evalExpr e2 s
6      | Expr.lt  e1 e2 => if evalExpr e1 s < evalExpr e2 s then 1 else 0
7
8      def update (s : State) (x : Var) (v : Int) : State :=
9      fun y => if y == x then v else s y
10
11     inductive BigStep : Stmt -> State -> State -> Prop
12     | assign : forall x e s,
13     BigStep (Stmt.assign x e) s (update s x (evalExpr e s))
14     | seq    : forall S1 S2 s s' s',
15     BigStep S1 s s' -> BigStep S2 s' s' -> BigStep (Stmt.seq S1 S2) s s'
16     | ifTrue : forall b S1 S2 s s',
17     evalExpr b s != 0 -> BigStep S1 s s' -> BigStep (Stmt.ifElse b S1 S2) s s'
18     | ifFalse : forall b S1 S2 s s',
19     evalExpr b s = 0 -> BigStep S2 s s' -> BigStep (Stmt.ifElse b S1 S2) s s'
20     | whileF : forall b S s,
21     evalExpr b s = 0 -> BigStep (Stmt.while b S) s s
22     | whileT : forall b S s s' s',
23     evalExpr b s != 0 -> BigStep S s s' -> BigStep (Stmt.while b S) s' s' ->
24     BigStep (Stmt.while b S) s s'

```

Lean 4

```

1      theorem determinism : forall S s s' s'',
2      BigStep S s s' -> BigStep S s s'' -> s' = s'' := by
3      intro S s s' s'' h1 h2
4      induction h1 generalizing s'' with
5      | assign =>
6      cases h2; rfl
7      | seq _ _ _ smid _ h1a h1b ih1 ih2 =>
8      cases h2 with
9      | seq h2a h2b =>
10     have hm := ih1 h2a
11     subst hm
12     exact ih2 h2b
13     | ifTrue _ _ _ _ _ hb _ ih =>
14     cases h2 with
15     | ifTrue _ h2 => exact ih h2
16     | ifFalse hb' _ => exact absurd hb' hb
17     | ifFalse _ _ _ _ _ hb _ ih =>
18     cases h2 with
19     | ifTrue hb' _ => exact absurd hb hb'
20     | ifFalse _ h2 => exact ih h2
21     | whileF _ _ _ hb =>
22     cases h2 with
23     | whileF _ _ => rfl
24     | whileT hb' _ _ => exact absurd hb hb'
25     | whileT _ _ _ smid _ hb _ _ ih1 ih2 =>
26     cases h2 with
27     | whileF hb' _ => exact absurd hb' hb
28     | whileT _ h2a h2b =>
29     have hm := ih1 h2a
30     subst hm
31     exact ih2 h2b

```

#### ۴.۵. وظیفه دانشجو

مثالی که در بخش قبل دیدید، یک نمونه کامل از اثبات قطعیت دستورات در معناشناسی ارزیابی بود تا با فضا آشنا شوید. حالا نوبت شماست. شما موظف هستید قضیه زیر را انتخاب و به طور کامل در یکی از محیط‌های Lean 4 یا Coq اثبات کنید:

**اثبات یکتایی و قطعیت ارزیابی عبارات (Expression Determinism):** ثابت کنید که ارزیابی عبارات هم قطعی، یعنی هر عبارت در هر state مشخص، دقیقاً به یک مقدار ارزیابی می‌شود:

$$\langle e, \sigma \rangle \Downarrow v \quad \wedge \quad \langle e, \sigma \rangle \Downarrow v' \quad \Rightarrow \quad v = v'$$

#### نکات مهم

اثبات شما باید در ابزار Lean 4 یا Coq بدون هیچ خطایی کامپایل بشود، یعنی نوشتن صرفاً ایده اثبات کافی نیست. علاوه بر کد، در گزارش توضیح دهید که منطق استقرای شما روی ساختار عبارت‌ها (Expression) چگونه پیش رفته و هر گام چه معنایی دارد. در جلسه ارائه باید بتوانید تک‌تک تاکتیک‌های اثبات خود را تشریح کنید.

موفق باشید!